

UNIVERSIDAD
POLITECNICA
DE VALENCIA

SISTEMAS INTEGRADOS DIGITALES

DISEÑO Y VERIFICACIÓN DE PERIFÉRICOS DESCRITOS EN VERILOG PARA PLATFORM DESIGNER

José María Monzó Ferrer

Valencia, 30 de abril de 2026

Versión 17.1, Platform Designer, DE1-SoC

CONTENIDOS

1. Objetivos 3
2. Introducción 3
3. Periférico: Decodificador hardware de hexadecimal a siete segmentos 3
4. Generación de un componente con interface Avalon-mm slave 7
5. Creación de un proyecto en Quartus II para un nuevo componente 8
6. Verificación funcional del componente desarrollado 10
6.1. Creación del elemento de librería de Platform Designer 10
6.2. Creación de un proyecto de verificación del componente en Platform Designer. 18
6.3. Verificación del componente haciendo uso del software Modelsim 22
7. Integración del nuevo componente en un Sistema en Nios II 28
8. TAREA: Diseño de un Timer propio en Verilog para Nios II 28
8.1. Descripción de los interfaces que poseerá el módulo 29
8.2. Descripción de la tarea, entregables y valoración 30

1. Objetivos

La finalidad del presente documento es que el alumno adquiera las destrezas necesarias para integrar el propio hardware digital diseñado en verilog como un periférico que formará parte de un sistema basado en Nios II definido en Platform Designer.

El alumno durante el ejercicio aprenderá a:

- Describir un interface estándar Avalon-MM slave.
- Conectar el interface Avalon-MM slave a un diseño propio definido en verilog generando un periférico compatible con un sistema basado en Nios II.
- Verificar el correcto funcionamiento del periférico desarrollado mediante técnicas avanzadas de verificación que hacen uso de modelos funcionales de bus (BFM) Avalon.
- Generar una librería de periféricos propios dentro de Platform Designer.
- Crear un sistema que haga uso de los nuevos periféricos desarrollados.
- Hacer uso del nuevo sistema generado programándolo en la placa DE1-SoC.

Finalizado el presente tutorial, el alumno dispondrá de las habilidades necesarias para poder mejorar sus sistemas digitales basados en Nios II mediante el uso de periféricos propios.

2. Introducción

En el presente tutorial, se guiará al alumno para generar un nuevo componente hardware para ser usado en un sistema basado en Nios II. En concreto, el alumno generará un bloque para acceder al display de siete segmentos de la DE1-SoC que incluya la decodificación de hexadecimal a siete segmentos vía hardware.

Posteriormente, se le solicitará el diseño de un hardware nuevo que se encargue de sustituir el elemento timer del sistema creando un nuevo timer programable con salida de un bit para detectar cruces por umbral también programable.

3. Periférico: Decodificador hardware de hexadecimal a siete segmentos

Hasta ahora, en los sistemas que hemos desarrollado, la decodificación de hexadecimal a siete segmentos para utilizar los displays disponibles en la DE1-SoC se llevaba a cabo vía software en el Nios II del sistema. Una forma de liberar a la CPU de esas tareas que consumen ciclos de CPU innecesariamente es trasladar esa funcionalidad a hardware.

Así pues, diseñando nuestro propio periférico decodificador de hexadecimal a siete segmentos en verilog podemos liberar a la CPU de ese cálculo y reducir la acción a una simple escritura en memoria que se encargará de trasladar el dato al hardware diseñado.

El diseño presentado posee tres registros de 32 bits mapeados en memoria. Los registros 0 y 1 son de escritura y permiten escribir en los displays de siete segmentos el dato a representar. El registro 2 es un ejemplo de registro de lectura para que el alumno pueda comprobar la forma de leer datos de un componente mediante un bus Avalon MM.

En el registro 0 están mapeados los displays de siete segmentos 0,1,2 y3:

reg0 tenemos {16’d0, HEX3,HEX2,HEX1,HEX0}

En el registro 1 están mapeados los displays de siete segmentos 4,5,6 y 7:

Reg1 tenemos {16’d0, HEX7,HEX6,HEX5,HEX4}

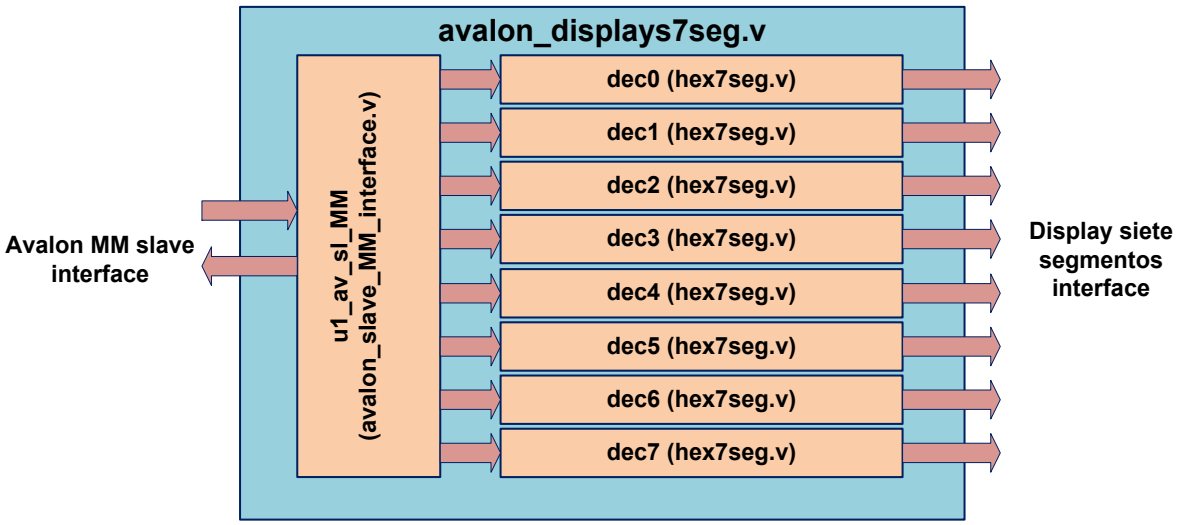


Figura 1. Estructura del periférico descrito.

En el presente documento, para facilitar la asimilación de la metodología a seguir, se le proporciona al alumno todo el código necesario del periférico.

El periférico se compone tres módulos descritos en verilog:

1. **avalon_displays7seg.v**: Es el top de la jerarquía, en él se instancian el interface con el bus avalon y los decodificadores. En sus puertos están las líneas correspondientes al interface Avalon-MM slave y las líneas correspondientes a las salidas que deberán conectarse a los displays de siete segmentos. El bloque posee un parámetro que nos permite invertir o no las salidas haciendo posible el uso del periférico para displays en cátodo común o en ánodo común.
2. **avalon_slave_MM_interface.v**: Este módulo se encarga de conectar nuestro decodificador de hexadecimal a siete segmentos con el bus avalon-MM. De esta forma, gracias a este interfaz, nuestro decodificador se comportará como un periférico Avalon-MM slave, permitiendo hacer uso de él a un microprocesador Nios II haciendo escrituras y lecturas en direcciones de memoria. Este módulo nos servirá como referencia para poder desarrollar nuestros propios interfaces Avalon-MM slave.
3. **hex7seg.v**: Es el módulo encargado de realizar la decodificación de hexadecimal a siete segmentos. Tiene un parámetro de entrada **invert** que permite invertir las salidas permitiendo de esta forma trabajar con displays en ánodo común o con displays en cátodo común. Como vemos en el top del diseño (**avalon_displays7seg.v**), de este módulo hacemos 8 instancias.

La estructura jerárquica aquí presentada independiza la funcionalidad del periférico, la decodificación, de la comunicación a través del interfaz avalon-MM, como puede verse en la **Figura. 1**. La idea es poder reutilizar el código de la interfaz Avalon-MM slave para próximos diseños y que cualquier error en la funcionalidad del decodificador no afecte a la comunicación. Recomiendo el uso de esta filosofía pues permite independizar las distintas funcionalidades del diseño haciendo más sencilla la etapa de validación y permitiendo el reúso de componentes en futuros diseños.

El código de los bloques del diseño:

avalon_displays7seg.v : Código del top de la jerarquía

```
module avalon_displays7seg(//Avalon MM interface signals
                           input reset,
                           input clock,
```

```
                              input chipselect,
                              input [2:0]  address,
                              input write,
                              input [31:0] writedata,
                              input read,
                              output [31:0] readdata,
                        //Conduit - Interface con 7 segmentos - salida externa.
                              output [6:0] HEX0,
                              output [6:0] HEX1,
                              output [6:0] HEX2,
                              output [6:0] HEX3,
                              output [6:0] HEX4,
                              output [6:0] HEX5,
                              output [6:0] HEX6,
                              output [6:0] HEX7);


parameter invert=0;

wire [31:0] reg1, reg0;


// Instancia del interface con el bus Avalon MM

avalon_slave_MM_interface u1_av_sl_MM (//Avalon MM interface signals
                                       .reset(reset),
                                       .clock(clock),
                                       .chipselect(chipselect),
                                       .address(address),
                                       .write(write),
                                       .writedata(writedata),
                                       .read(read),
                                       .readdata(readdata),
                                      //Interface con nuestra logica.
                                        // Registros 0 y 1 de lectura
                                       .reg0(reg0),
                                       .reg1(reg1),
                                        //Datos para el registro interno reg3
                                       .data(32'h87654321),
                                       .we(1'b1)); // Write enabl. de reg3



hex7seg #(.invert(invert)) dec0 (.inhex(reg0[3:0]),   .out7seg(HEX0));
hex7seg #(.invert(invert)) dec1 (.inhex(reg0[7:4]),   .out7seg(HEX1));
hex7seg #(.invert(invert)) dec2 (.inhex(reg0[11:8]),  .out7seg(HEX2));
hex7seg #(.invert(invert)) dec3 (.inhex(reg0[15:12]), .out7seg(HEX3));

hex7seg #(.invert(invert)) dec4 (.inhex(reg1[3:0]),   .out7seg(HEX4));
hex7seg #(.invert(invert)) dec5 (.inhex(reg1[7:4]),   .out7seg(HEX5));
hex7seg #(.invert(invert)) dec6 (.inhex(reg1[11:8]),  .out7seg(HEX6));
hex7seg #(.invert(invert)) dec7 (.inhex(reg1[15:12]), .out7seg(HEX7));


endmodule
```

Código verilog del módulo avalon_displays7seg.v

avalon_slave_MM_interface.v: Código del interface con el bus Avalon, que hace que nuestro diseño se comporte como un Avalon-MM-slave

```
module avalon_slave_MM_interface(//Avalon MM interface signals
                                 input reset,
                                 input clock,
                                 input chipselect,
                                 input [2:0]  address,
                                 input write,
                                 input [31:0] writedata,
                                 input read,
                                 output reg [31:0] readdata,
                                 //Interface con nuestra logica.
                                   // Registros 0 y 1 de lectura
                                 output reg [31:0] reg0, reg1,
                                   // Datos para el registro interno reg3
                                 input  [31:0] data,
                                   // Write enable para el registro interno.
                                 input   we);


// Componente con 3 registros mapeados en memoria reg0, reg1 y reg2
// reg0 y reg1 son registros de lectura y escritura
// reg2 es un registro de solo lecutura
reg [31:0] reg2;
always @(posedge clock)
    if (reset)
    begin
          reg0 <= 32'd0;
          reg1 <= 32'd0;
          reg2 <= 32'd0;
          readdata <= 32'd0;
     end
     else
    begin
          if (chipselect)
          begin
              // Proceso de escritura en el interface Avalon MM
              if (write)
               begin
                    case(address)
                      3'd0: reg0 <= writedata;
                      3'd1: reg1 <= writedata;
                     endcase
              end
              // Proceso de lectura del interface Avalon MM
              if (read)
              begin
                    case(address)
                    3'd0: readdata <= reg0;
                    3'd1: readdata <= reg1;
                    3'd2: readdata <= reg2;
                    // Si la direccion es incorrecta devuelvo todo ceros.
                   default: readdata <= 32'd0;
                   endcase
              end
              // Proceso de escritura en el registro 2
           end
           if (we) reg2 <= data;

    end
endmodule
```

Código verilog del módulo avalon_slave_MM_interface.v

hex7seg.v: Código del decodificador de hexadecimal a siete segmentos. Este módulo transforma un número hexadecimal de 4 bits en un código representable en display de siete segmentos.

```
module hex7seg(input [3:0] inhex,
        output [6:0] out7seg);
parameter invert=0;
reg [6:0] out7seg_aux;
always @(*)
    case (inhex)
     4'h0: out7seg_aux = 7'h3F;
     4'h1: out7seg_aux = 7'h06;
     4'h2: out7seg_aux = 7'h5B;
     4'h3: out7seg_aux = 7'h4F;
     4'h4: out7seg_aux = 7'h66;
     4'h5: out7seg_aux = 7'h6D;
     4'h6: out7seg_aux = 7'h7D;
     4'h7: out7seg_aux = 7'h07;
     4'h8: out7seg_aux = 7'h7F;
     4'h9: out7seg_aux = 7'h67;
     4'hA: out7seg_aux = 7'h77;
     4'hB: out7seg_aux = 7'h7C;
     4'hC: out7seg_aux = 7'h39;
     4'hD: out7seg_aux = 7'h5E;
     4'hE: out7seg_aux = 7'h79;
     4'hF: out7seg_aux = 7'h71;
     default: out7seg_aux = 7'h7F;
     endcase
assign out7seg=invert? ~(out7seg_aux): out7seg_aux;

endmodule
```

Código verilog del módulo hex7seg.v

4. Generación de un componente con interface Avalon-mm slave

La generación de un componente se llevará a cabo en distintas fases:

1. Crearemos un proyecto en Quartus II donde diseñaremos el periférico en verilog.
2. Diseñado el componente, deberemos generar un elemento de librería en Platform Designer.
3. Crearemos un proyecto de verificación en Platform Designer.
4. Verificaremos el diseño haciendo uso de Modelsim.
5. Usaremos el componente dentro de un sistema basado en Nios II.

Durante la verificación, tendremos problemas de diseño que deberemos resolver. Para ello, volveremos al paso 2 anterior y corregiremos los errores. En el paso 3, actualizaremos el componente de la librería con los nuevos cambios y volveremos a verificar el componente. Este proceso será iterativo hasta que consideremos el componente totalmente verificado. Por tanto, trabajaremos en paralelo con tres herramientas al mismo tiempo, Quartus II, Platform Designer y Modelsim, para poder crear el componente, generar un periférico en Platform Designer y verificarlo.

Verificado el componente podemos considerarlo útil para ser usado en sistema basado en Nios II.

5. Creación de un proyecto en Quartus II para un nuevo componente

Para desarrollar un componente que sea sintetizable y, por tanto, válido como elemento hardware, vamos hacer uso de un proyecto en Quartus II. Seguiremos los siguientes pasos:

1. Generaremos un proyecto en Quartus II, ya que en un futuro vamos a usar el componente en un sistema basado en Nios II que correrá en una placa DE1-SoC. Seleccionaremos la FPGA de dicha placa.
2. Desarrollaremos todo el código verilog, los distintos módulos que componen nuestro periférico.
3. Comprobaremos que no posee ningún error de sintaxis.
4. Comprobaremos que el diseño es sintetizable.

En esta parte de la práctica, el alumno hará uso del código verilog del periférico a desarrollar proporcionado anteriormente.

Para definir y crear el sistema integrado propuesto, se comienza ejecutando el software de Altera Quartus II y, a continuación, se seguirán los siguientes pasos:

1 Crear un nuevo proyecto Quartus II para el desarrollo del componente. Tal como se muestra en la Figura 2., se almacena el nuevo proyecto en un directorio denominado `avalon_displays7seg`, y se le da el nombre de `avalon_displays7seg`, tanto al proyecto como a la entidad jerárquica superior del diseño. Cada uno puede escoger el directorio o el nombre de proyecto que desee. En este proyecto se escogerá el dispositivo de la familia Cyclone V **5CSEMA5F31C6,** ya que es la FPGA que tiene la placa DE1-SoC.

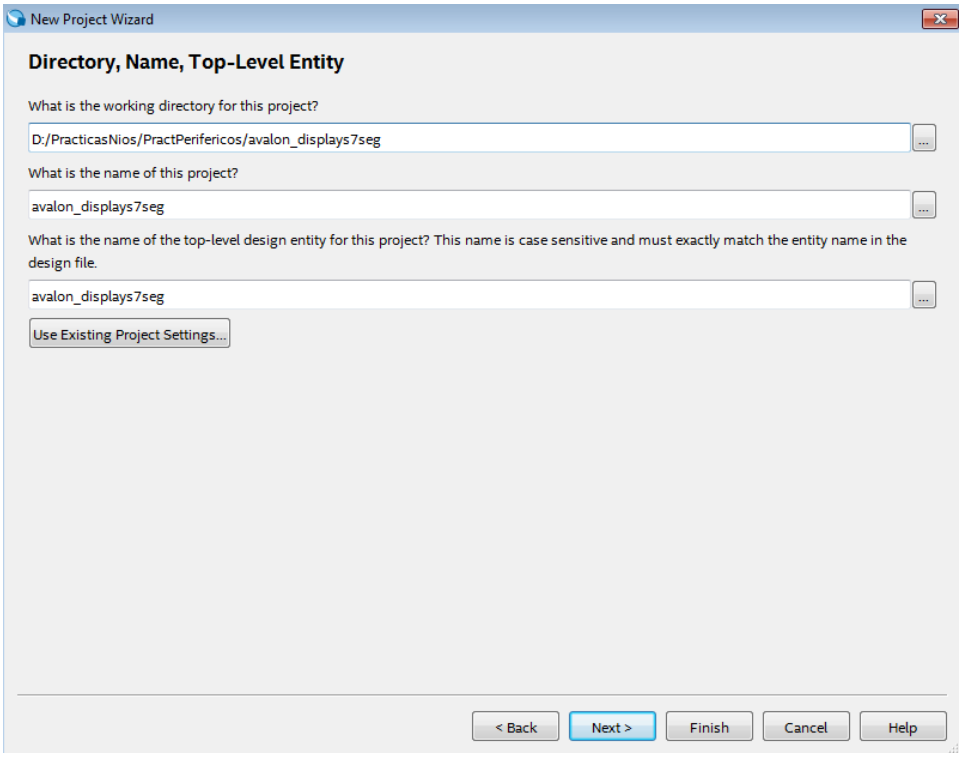


Figura 2. Crear un nuevo proyecto.

2 Si abrimos un explorador de archivos podremos observar que, en nuestro caso, en "**D:\PracticasNios\PractPerifericos\avalon_displays7seg**" se ha creado el proyecto de Quartus II donde diseñaremos nuestro componente. Dentro de esta carpeta del proyecto,

creamos una carpeta **display7seg** donde guardaremos el hdl de nuestro periférico. Por tanto, en la carpeta "**D:\PracticasNios\PractPerifericos\avalon_displays7seg\display7seg**" es donde guardaremos la nueva IP. En esta ocasión, vamos a copiar directamente los tres ficheros proporcionados tal como quedan en la imagen **Figura 3**.

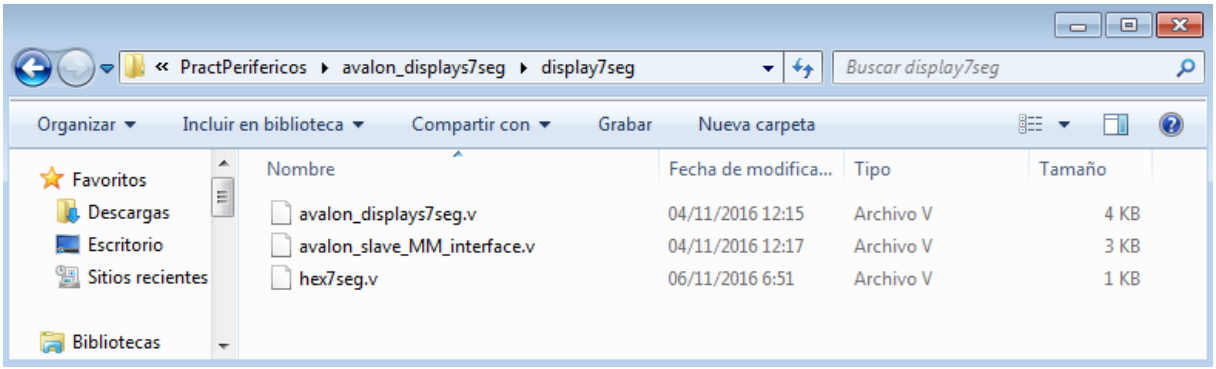


Figura 3. Directorio con los ficheros del diseño.

3 Ahora añadiremos al proyecto los ficheros verilog que hemos copiado. Para ello, seleccionamos la opción **Project > Add/Remove Files In Project …** donde indicaremos los nuevos ficheros como indica la **Figura 4.**

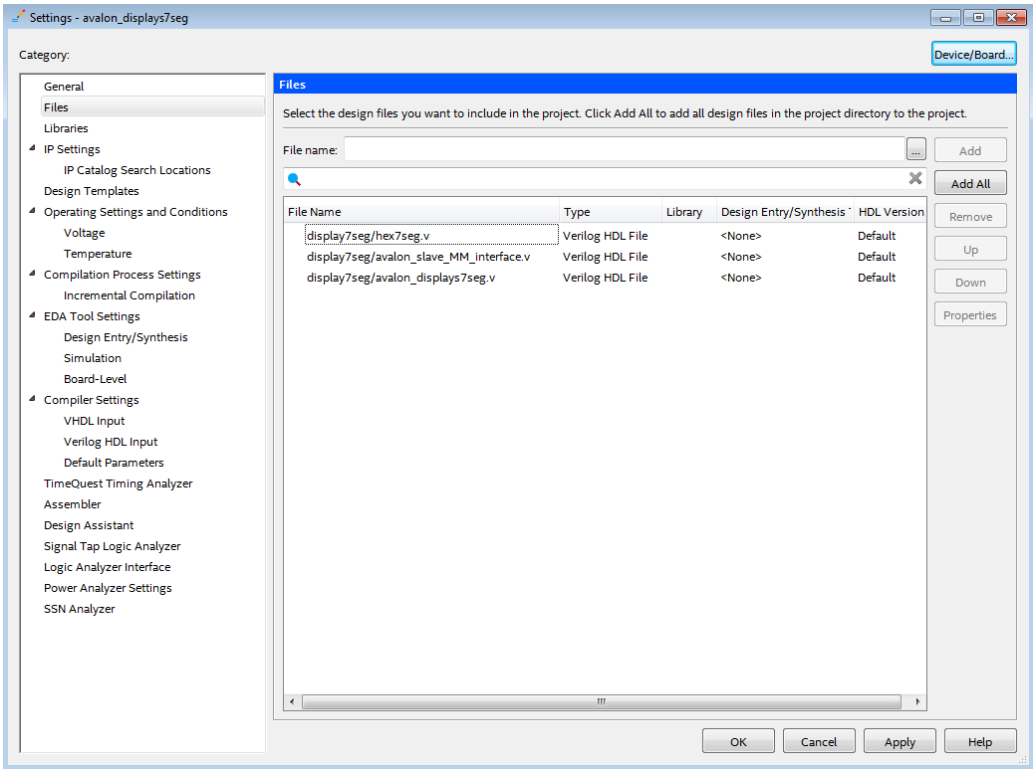


Figura 4. Ficheros incluidos en el proyecto.

4 Si no hemos cometido ningún error a la hora de nombrar el fichero top de la jerarquía cuando creamos el proyecto, automáticamente tendremos ya el diseño con el módulo avalon_displays7seg como top. Podemos ahora, comprobar si el diseño puede compilarse. En el menú **Processing,** hacer clic en **Start Compilation**.

5 En la ventana de Quartus II de Status se visualiza el progreso. La compilación puede tardar algunos minutos. Cuando la compilación termina, aparece una ventana con el mensaje: `Full compilation was successful`.

6 Hacer clic en **OK**. En Quartus II se mostrará la ventana del informe de compilación.

En este momento, tenemos un diseño sin errores sintácticos y que es sintetizable. Podemos observar en la ventana **Flow Summary** un resumen del uso de recursos de la FPGA del componente diseñado. En esta fase del diseño tenemos que validar si el diseño no posee errores a nivel funcional. Para ellos tendremos que desarrollar una validación funcional del componente antes de integrarlo dentro de un sistema con Nios II.

6. Verificación funcional del componente desarrollado.

El objetivo de la verificación funcional es poder validar el correcto funcionamiento del componente mediante software de verificación sin tener que hacer uso de un sistema Nios II completo. De esta forma, de forma rápida y controlada generaremos distintos casos de simulación que nos permiten verificar que nuestro componente diseñado se comporta como se espera dentro de un sistema basado en bus Avalon-MM.

Para poder validar el sistema sin hacer uso de un Nios II, haremos uso de modelos funcionales de bus (BFM). Los BFM son componentes de verificación disponibles en la librería de Platform Designer que permiten emular transacciones de bus Avalon-MM a nivel de testbench mediante el uso de funciones. Así pues, si nuestro diseño posee un interface Avalon-MM slave, necesitaremos de un BFM Avalon-MM master que, como veremos, es el elemento de la librería de Platform Designer con el nombre **Avalon MM Master BFM.** Haciendo uso de estos componentes, podemos asegurarnos que las transacciones Avalon MM siguen el standard del bus.

Así pues, para realizar la verificación funcional, realizaremos los siguientes pasos:

1. Crear un elemento de librería en Platform Designer.
2. Crear un proyecto de verificación en Platform Designer.
3. Verificar el diseño haciendo uso de Modelsim.

6.1. Creación del elemento de librería de Platform Designer

7 Desde el proyecto en Quartus II, arrancamos Platform Designer desde la opción **Tools > Platform Designer**. De esta forma, arrancamos Platform Designer, cuya pantalla inicial puede verse en la Figura 5.

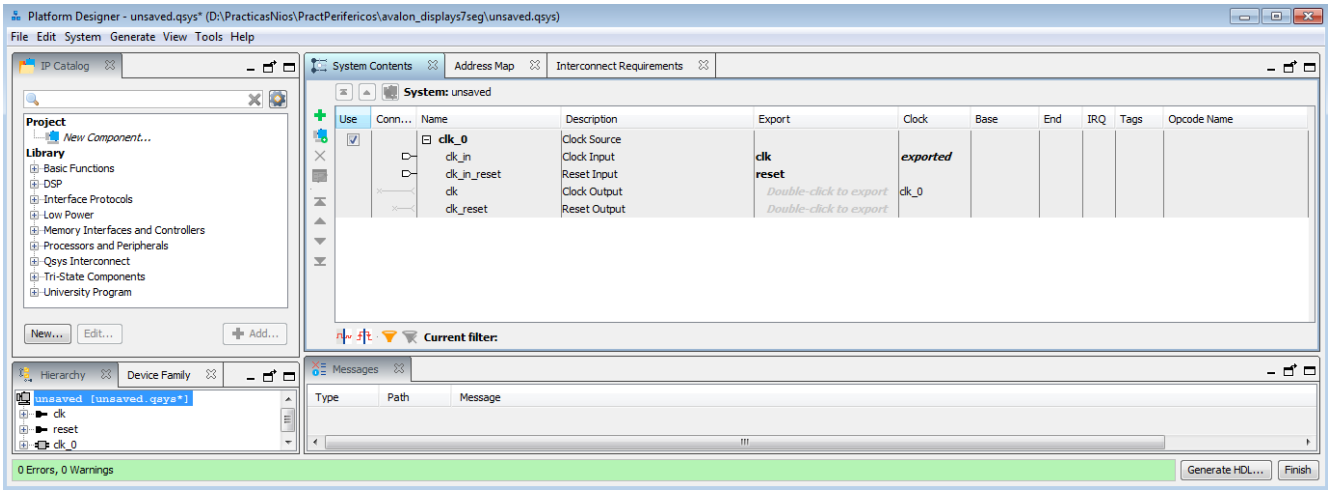


Figura 5. Pantalla inicial de Platform Designer.

8 Guardamos el proyecto Platform Designer, nos vamos a **File > Save As…** y guardamos el proyecto Platform Designer con el nombre **avalon_displays7seg_qsys.qsys .** El nombre

seleccionado aquí es arbitrario, pero es importante porque determina la ruta de los ficheros que se generarán posteriormente para realizar la verificación.

9 Ahora pasamos a la creación del nuevo componente para la librería de Platform Designer. Pulsamos en **Component Library > Project > New Component …** Aparece la ventana de Edición de componentes, concretamente la pestaña **Component Type**. En esa pestaña definimos:

1. Nombre del componente (Name): **avalon_displays7seg**
2. Nombre con el que aparecerá el componente (Display Name): **avalon_displays7seg**
3. Nombre de la librería donde se va a almacenar (Group): **myAvalonIPs** . Si la librería no existe, se creará nueva.

En esta pestaña podríamos añadir hojas de catálogo, descripción de componente, etc.

Así pues, la pestaña quedará cuando la rellenemos como la que puede verse en la Figura 6.

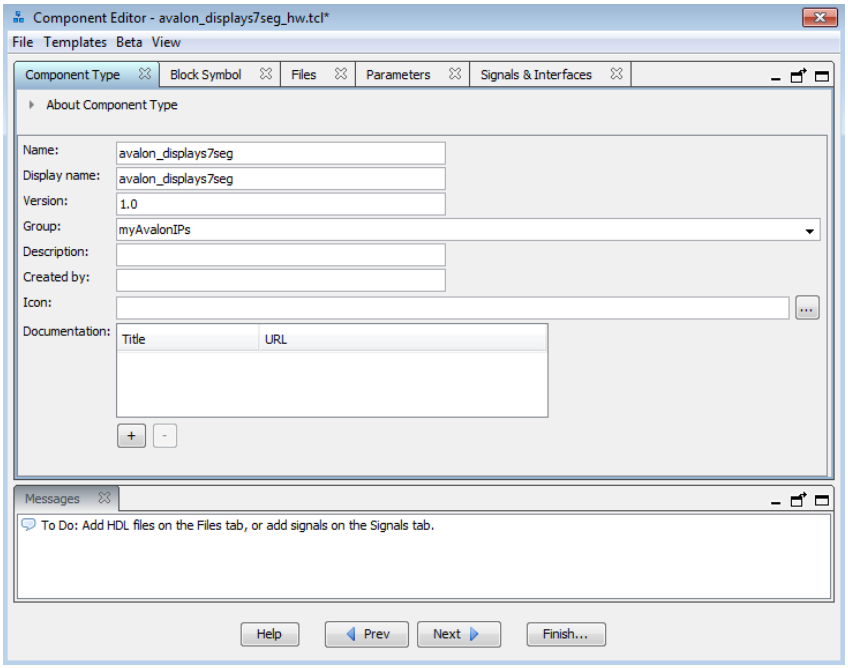


Figura 6. Editor de componentes pestaña Component Type.

10 Pasamos a la pestaña **Files** del editor de componentes**.** En esta pestaña, indicamos los ficheros en verilog que definen el componente tanto para síntesis como para simulación. En **Synthesis Files,** pulsando el botón **Add File …,** añadimos los tres ficheros verilog diseñados en el proyecto de Quartus II. Una vez añadidos los ficheros, tendremos que indicar cuál es el top de la jerarquía de nuestro componente. Para ello, en la columna de **Attributes** del fichero **avalon_displays7seg.v** de la tabla de **Synthesis Files,** podemos indicar que este es el módulo top de nuestro componente.

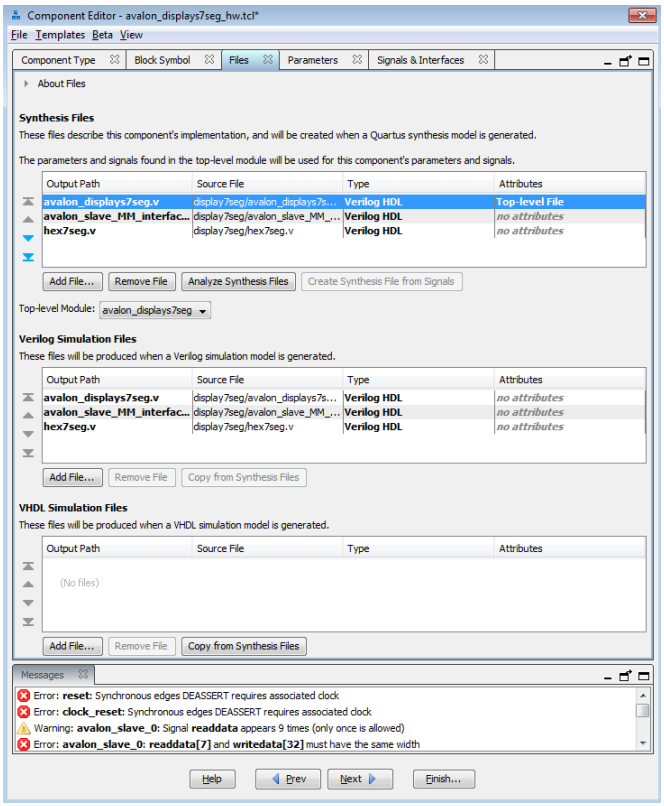


Figura 7: Editor de componentes pestaña Files

Tras esto, analizamos los ficheros con el botón **Analyze Synthesis Files.** Como los ficheros ya fueron analizados en Quartus II y no dieron errores, en esta ocasión tampoco aparecerán errores de síntesis.

Si observamos el panel de información situado en la parte inferior del editor de componentes, vemos que tras la síntesis han aparecido errores de la declaración del componente. Estos errores irán desapareciendo a medida que vayamos pasando por el proceso de creación del componente.
Para finalizar con la pestaña **Files,** es importante indicar cuáles van a ser los ficheros de simulación. Como nuestro componente está descrito en verilog, en la sección **Verilog Simulation Files** pulsaremos el botón **Copy From Synthesis Files,** que se encarga de importar los mismos ficheros de simulación ya usados para la síntesis.
La pestaña queda como vemos en la Figura 7.

11 La pestaña **Parameters** contiene los parámetros definidos en el fichero verilog top del dispositivo. Permite indicar el valor por defecto de los parámetros y si son editables cuando se instancie el componente en un sistema. En nuestro caso, nuestro componente dispone de un parámetro para indicar si las salidas a siete segmentos están invertidas o no para poder atacar a displays en cátodo común o en ánodo común. En la Figura 8 se muestra cómo queda configurado el panel.

Este panel permite comprobar cómo se visualizará el componente cuando vaya a ser instanciado en un proyecto en Platform Designer pulsando el botón **Preview the GUI...**

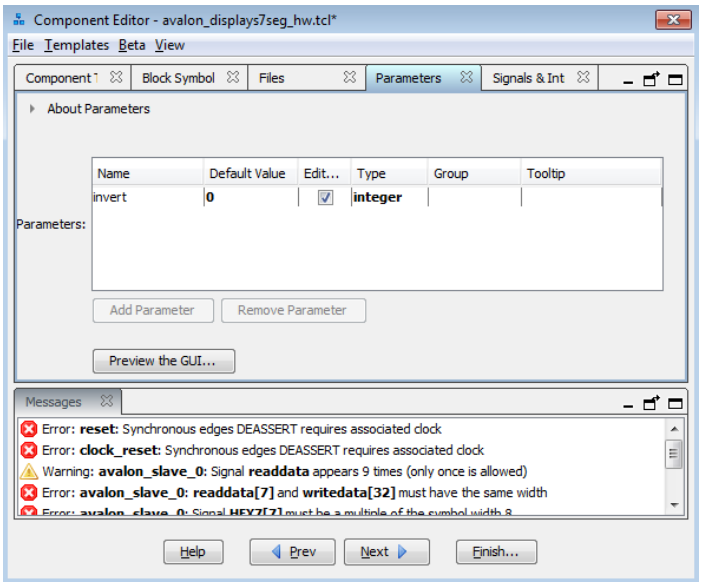


Figura 8: Editor de componentes pestaña Parameters

12 En la pestaña **Signals & Interfaces,** aquí vamos a ir definiendo los interfaces que tiene el componente. También, una vez definidos los interfaces, podemos vincular cada puerto del periférico con el interfaz al que pertenece. Esta vinculación de las señales con la señal del interface a la que corresponden, en esta práctica, no la realizaremos en esta pestaña.

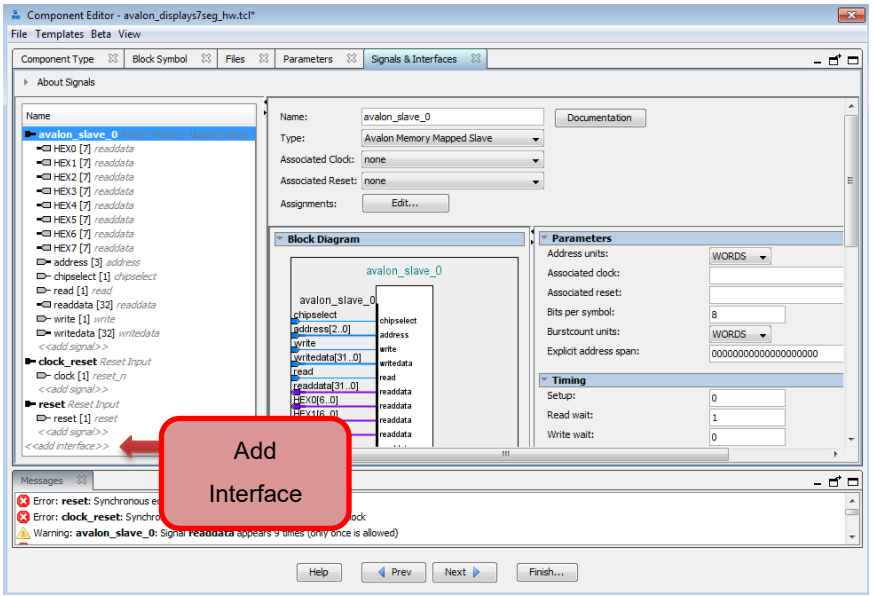


Figura 9: Editor de componentes pestaña Signals & Interfaces

En nuestro diseño, vamos a necesitar los siguientes interfaces:

1. Interface de tipo **Clock input**: Para el reloj del componente.
2. Interface de tipo **Reset input**: Para el reset del componente.
3. Interface de tipo **Avalon Memory Maped Slave**: Que agrupará las líneas que definen la comunicación a través del bus Avalon-MM
4. Interface de tipo **Conduit:** Que agrupará las líneas que salen al exterior del sistema de Platform Designer, que en nuestro componente son las líneas que serán conectadas a los displays de siete segmentos.

Para añadir un interface, disponemos de un botón en la pestaña **Interfaces** denominado **Add Interface** Figura 9. Por defecto Platform Designer, ha añadido algunos interfaces considerando que nuestras señales pertenecían a ellos. Nosotros los modificaremos e introduciremos los que necesitamos. Luego, una vez definidos correctamente todos los puertos, eliminaremos los interfaces no necesarios.

13 Necesitamos un **Clock input** interface (Figura 10):

Modificamos uno de los interfaces para que cumpla lo siguiente:

1. **Name:** clock_sink
2. **Type:** Clock input

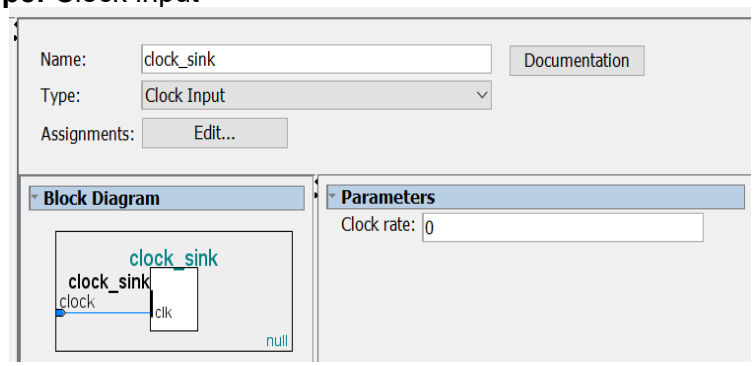


Figura 10: Definición interface de reloj del componente

14 Necesitamos un **Reset input** interface (Figura 11):

Modificamos uno de los interfaces para que cumpla lo siguiente:

1. **Name:** reset_sink
2. **Type:** Reset input
3. **Associated Clock:** clock_sink
4. **Synchronos Edges:** Both

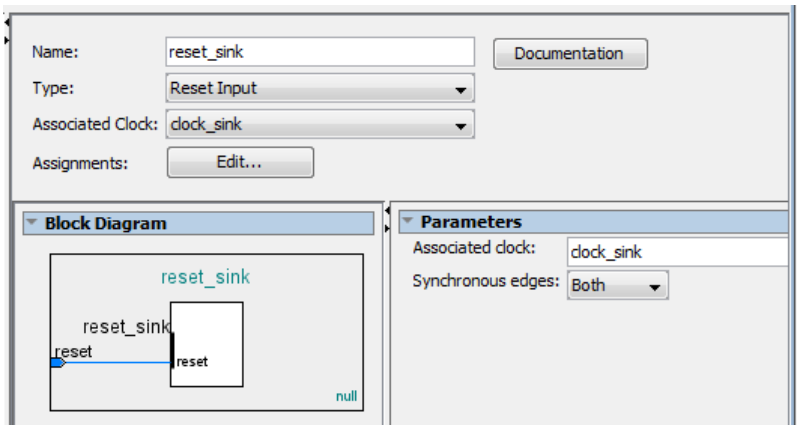


Figura 11: Definición interface de reset del componente

15 Necesitamos un **Avalon Memory Mapped Slave** interface (Figura 12):

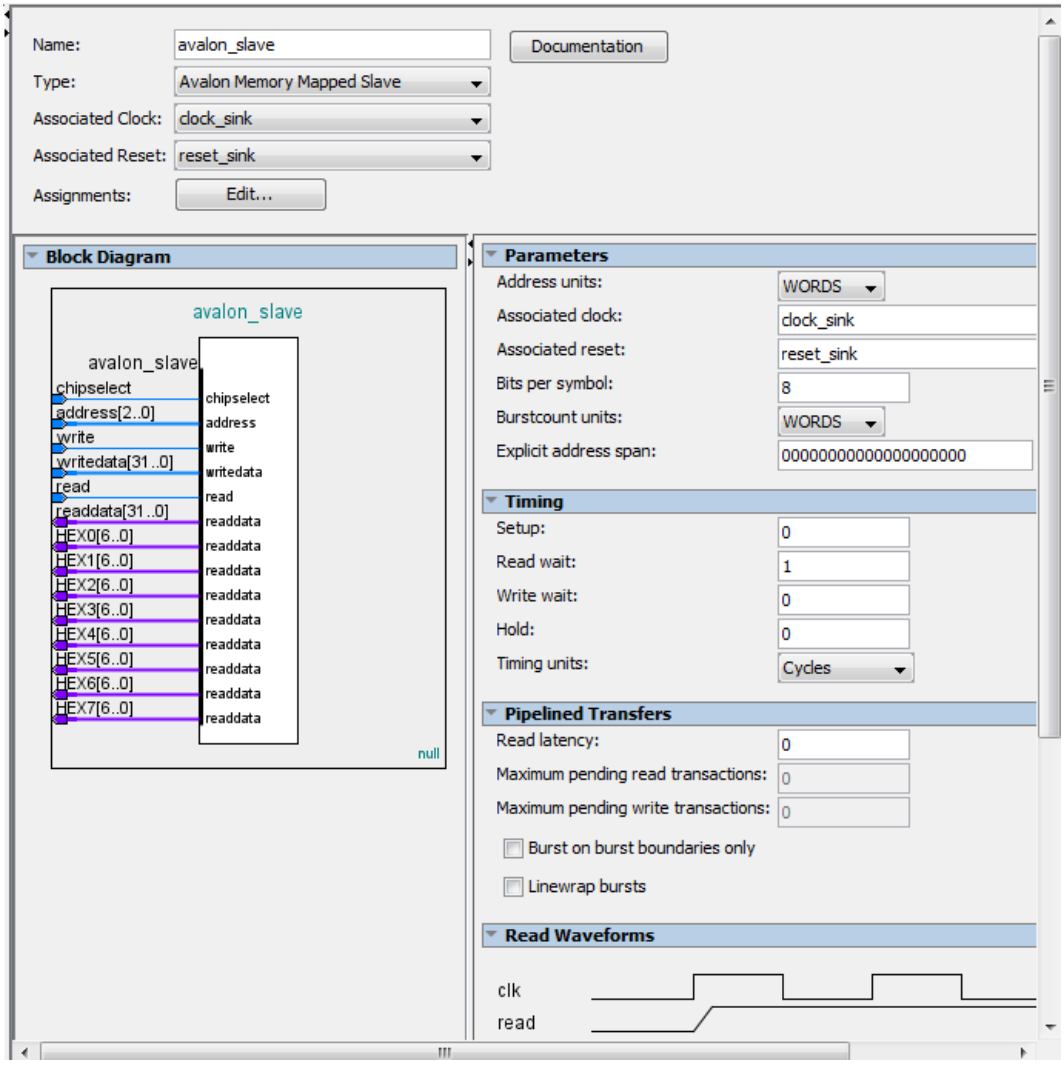


Figura 12: Definición interface Avalon-MM slave

Modificamos uno de los interfaces para que cumpla lo siguiente:

1. **Name:** avalon_slave
2. **Type:** Avalon Memory Mapped Slave
3. **Associated Clock:** clock_sink
4. **Associated Reset:** reset_sink

Parameter:

- **Address units**: WORDS
- **Bits per symbol**: 8

Timing:

- **Setup**: 0
- **Read wait**:1
- **Write wait**: 0
- **Hold**: 0
- **Timing units**: Cycles

Pipelined Transfers:

- **Read latency**: 0
- **Burst on burst boundaries only**: Desactivado
- **Linewrap bursts**: Desactivado

Con esto definimos, un interface Avalon-MM slave que se ajusta al definido en nuestro código verilog, con 0 ciclos de tiempo de respuesta en escritura y con un tiempo de respuesta de 1 ciclo en lectura. Como puede verse en la definición del interface en la Figura 12, el editor proporciona un ejemplo de cronograma de la escritura y lectura que nos puede ayudar a comprobar si se ajusta al que hemos definido en el verilog.

16 Por último, necesitamos crear un interface de tipo **Conduit**. Es el interface que usamos para la señales que van hacia el exterior del sistema Platform Designer (Figura 13):

Modificamos uno de los interfaces para que cumpla lo siguiente:

1. **Name:** external_interface
2. **Type:** Conduit
3. **Associated Clock:** clock_sink
4. **Associated Reset:** reset_sink

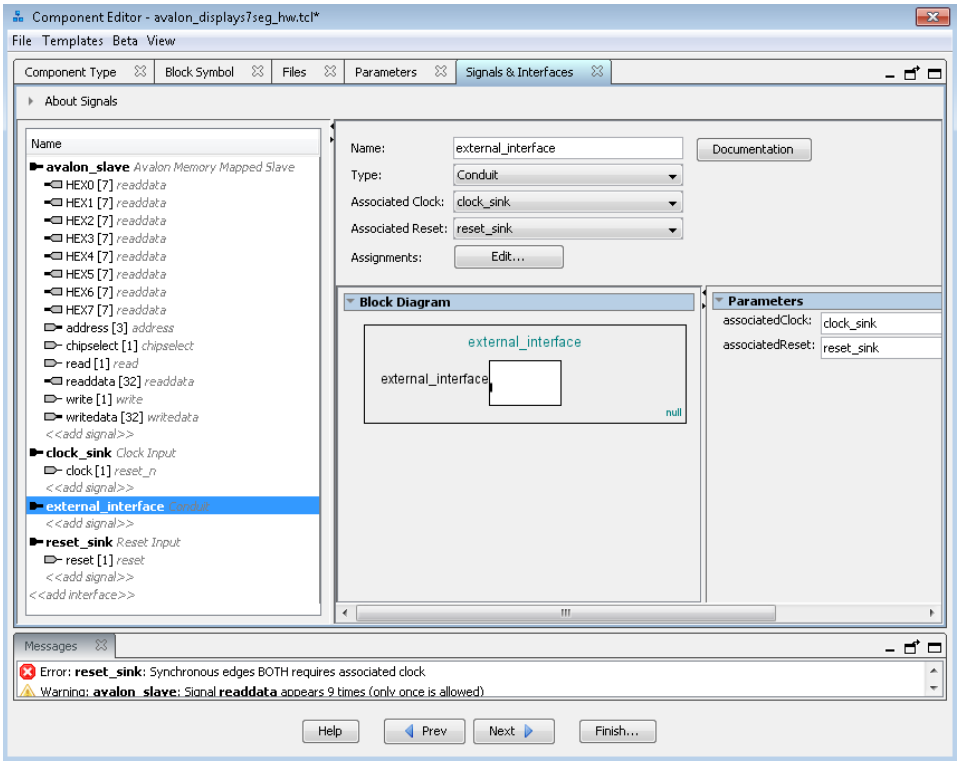


Figura 13: Definición interface conduit

17 Ya hemos definido todos los interfaces que necesitamos para nuestro componente (Figura 13). Ahora tenemos que vincular cada uno de los puertos de entrada y salida del componente con sus correspondientes señales de cada uno de los interfaces. Esto se podría hacer en la pestaña **Signals & Interfaces** pero es más cómodo vincularlas en la pestaña **Signals** que por defecto está oculta. Para visualizar la pestaña hacemos en **View > Signals.** En esta pestaña vinculamos las señales de mi periférico con las señales de los interfaces. Su configuración debe quedar como en la Figura 14:

Interface reset_sink:

- **Señal**: Reset; **Interface**: reset_sink**; Signal Type** :reset

Interface clock_sink

- **Señal**: clock; **Interface**: clock_sink**; Signal Type**:clk

Interface avalon_slave

- **Señal**: chipselect; **Interface**: avalon_slave**; Signal Type**: chipselect

- **Señal**: address; **Interface**: avalon_slave**; Signal Type**: address
- **Señal**: write; **Interface**: avalon_slave**; Signal Type**: write
- **Señal**: writedata; **Interface**: avalon_slave**; Signal Type**: writedata
- **Señal**: read; **Interface**: avalon_slave**; Signal Type**: read
- **Señal**: readdata; **Interface**: avalon_slave**; Signal Type**: readdata

Interface external_interface

- **Señal**: HEX0; **Interface**: external_interface; **Signal Type**: conduit0.
- **Señal**: HEX1; **Interface**: external_interface; **Signal Type**: conduit1.
- **Señal**: HEX2; **Interface**: external_interface; **Signal Type**: conduit2.
- **Señal**: HEX3; **Interface**: external_interface; **Signal Type**: conduit3.
- **Señal**: HEX4; **Interface**: external_interface; **Signal Type**: conduit4.
- **Señal**: HEX5; **Interface**: external_interface; **Signal Type**: conduit5.
- **Señal**: HEX6; **Interface**: external_interface; **Signal Type**: conduit6.
- **Señal**: HEX7; **Interface**: external_interface; **Signal Type**: conduit7.

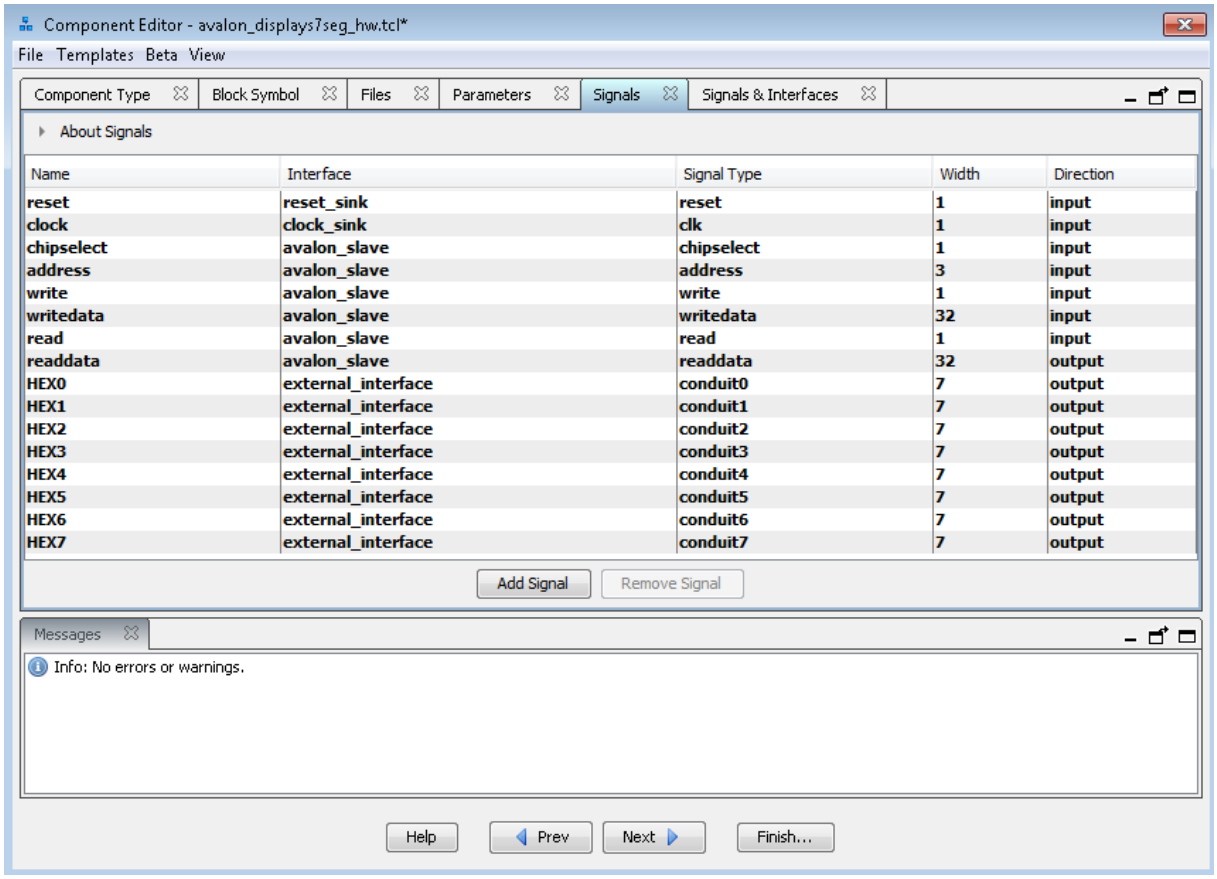


Figura 14: Editor de componentes pestaña Signals

Como puede observarse en la Figura 14, una vez definidos correctamente todos los puertos, en la parte inferior de la pantalla en el panel de información han desaparecido todos los errores. Ahora el componente ya está listo para ser guardado en la librería de Platform Designer.

18 Pulsamos el botón **Finish…** y el componente es guardado en la librería **myAvlonIPs** que hemos creado. Este componente ya está listo para ser usado en un proyecto Platform Designer como cualquier otro componente de la librería. Podemos ver en la Figura 15 como queda ahora la pestaña **Component Library**

Si queremos realizar cualquier cambio sobre el componente para añadir funcionalidad o corregir fallos encontrados, colocándonos con el botón derecho del ratón podemos ejecutar **Edit…** y volvemos a ejecutar el editor de componentes.

No olvidemos que para que los cambios se efectúen sobre un sistema que ya esté usando este componente deberemos hacer **File > Refresh System**

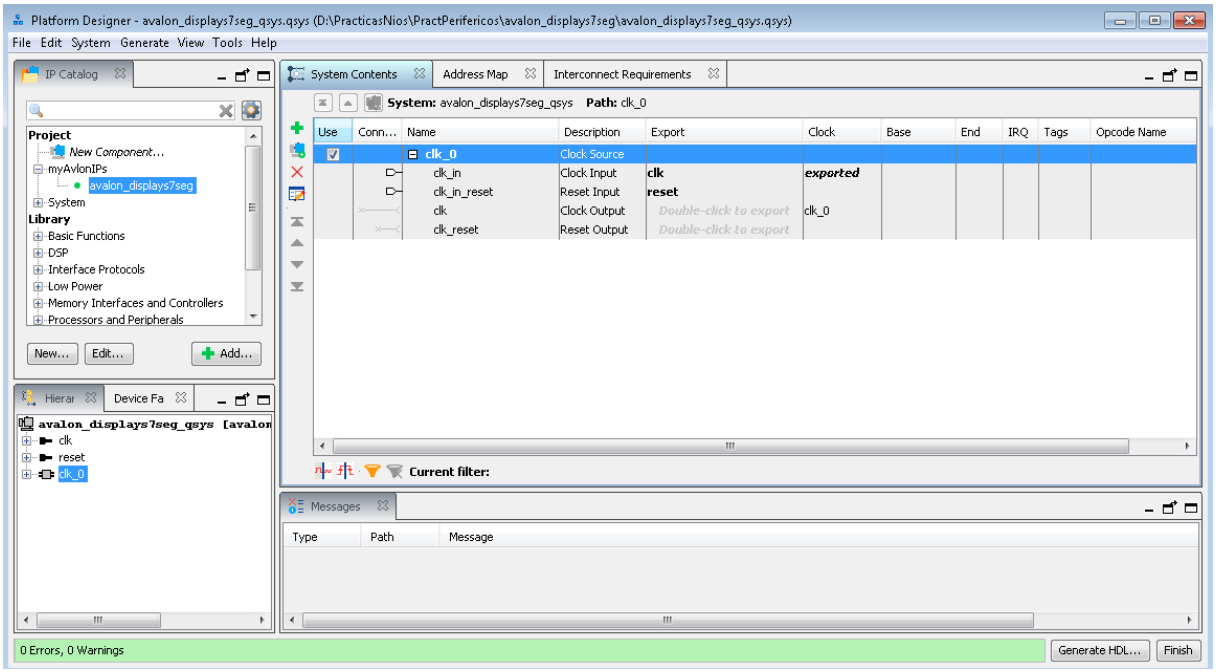


Figura 15: Componente creado ya en la nueva librería

19 Guardar los cambios **File > Save.** Ya tenemos nuestro componente como elemento de librería para ser utilizado en un sistema Platform Designer con bus Avalon MM.

6.2. Creación de un proyecto de verificación del componente en Platform Designer.

Tenemos ya nuestro componente en una librería de Platform Designer. El componente es plenamente sintetizable pero no lo hemos verificado a nivel funcional. En este apartado veremos cómo crear un proyecto en Platform Designer para verificar nuestro nuevo componente. La idea es no usar un microprocesador Nios II para verificar el componente pues ralentizaría mucho la simulación.

Emularemos el acceso al componente a través del bus Avalon-MM mediante un BFM (Modelo Funcional de Bus). Como nuestro dispositivo se comporta como un Avalon MM slave, en este apartado veremos cómo crear un proyecto con nuestro componente y un BFM Avlon-MM master que genere transacciones hacia nuestro diseño.

Creado el sistema, generaremos los ficheros de simulación que deberán ser modificados para poder realizar la simulación en Modelsim, proceso que veremos en el apartado siguiente.

Así pues, el proceso para crear un sistema Platform Designer verificable que contenga nuestro diseño es el siguiente:

20 Por defecto, el sistema que tenemos tiene únicamente el componente generador de reloj y reset. Tendremos ahora que añadir nuestro diseño:

- Seleccionar nuestro componente: **Component Library > Project > myAvalonIPs > avalon_displays7seg** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- En la ventana de configuración de este componente, configuramos el Parameter **invert a 1** para que funcione con la polaridad correcta para los displays LED de la placa DE1-SoC
- Pulsando en **Finish,** se volverá a la ventana de contenidos del sistema.
- Renombrar el componente como *displays7seg*.
- Conectar el puerto `clk_sink` del *displays7seg* con el puerto `clk` del *clk_0*.
- Conectar el puerto `rst_sink` del *displays7seg* con el puerto `clk_rst` del *clk_0*.
- En la fila `external_interface` del *displays7seg*, hacer click en la columna **Export** para exportar los puertos.

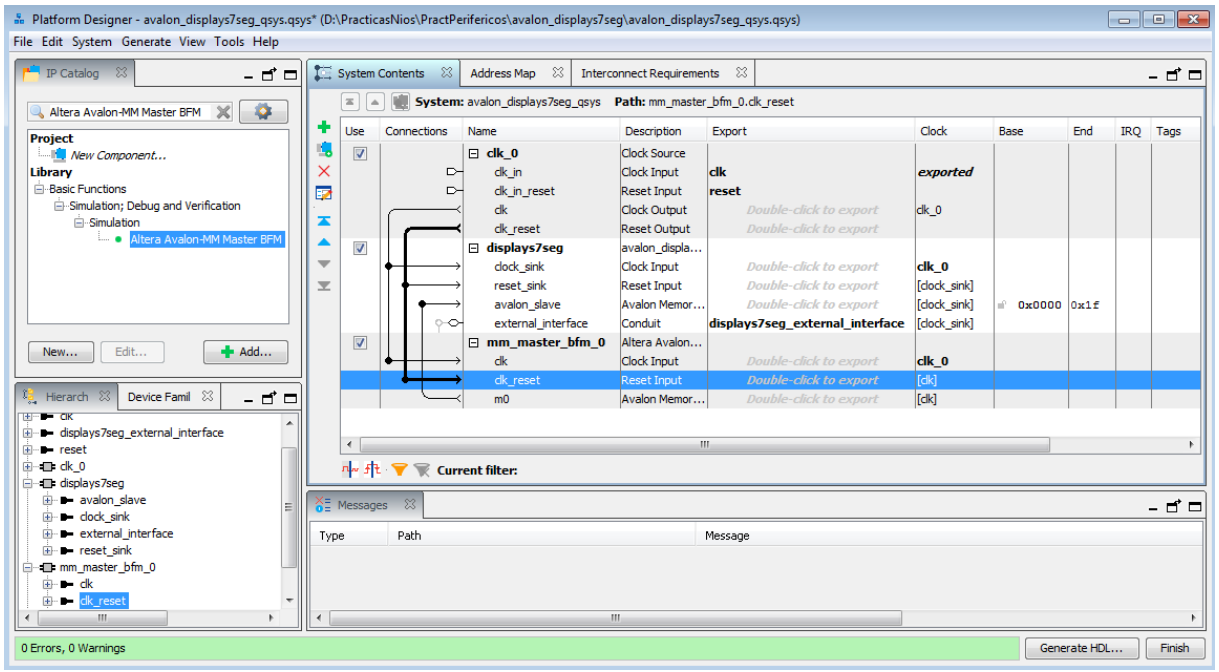


Figura 16: Systema Platform Designer para verificar el componente desarrollado

21 Para poder acceder a nuestro diseño a través del bus Avalon-MM, tenemos que instanciar el BFM del Avalon-MM master que nos permita conectarlo al Avalon-MM slave de nuestro diseño:

- Seleccionar el BFM: **Component Library > Library > Basic Functions > Simulation; Debug and Verification > Altera Avalon-MM Master BFM** y pulsar en **Add**, para lanzar la ventana de configuración del componente.
- En la ventana de configuración de este componente configuramos el componente para que sea compatible con la configuración Avalon MM slave introducida en nuestro diseño:

Ports Widths

- **Adress width** : 3 (bits usados por nuestro componente, mínimo que no da problemas)
- **Symbol width**: 8

Parameters

- **Number of symbols**: 4
- **Burstcount Witdh**: 1

Port Enables

- **Use the read signal: Activado**
- **Use the write signal: Activado**
- **Use the address signal: Activado**
- Use the byteeneable signal: Desactivado
- Use the burstcount signal: Desactivado
- **Use the readdata signal: Activado**
- Use the readdatavalid signal: Desactivado
- **Use the writedata signal: Activado**
- Use the begintransfer signal: Desactivado
- Use beginbursttransfer signal: Desactivado
- Use the arbiterlock signal: Desactivado
- Use the lock signal: Desactivado
- Use the debugaccess signal: Desactivado
- **Use the waitrequest signal: Activado** (Esta señal no está en nuestro diseño porque nuestro componente trabaja con tiempos de respuesta constantes. Si no se introduce en el BFM, este da problemas. Que esté la señal no afecta a la comunicación con el BFM)
- Use the clken signals: Desactivado

Port polarity

TODAS ACTIVADAS

Burst Attributes

TODOS DESACTIVADOS

Miscellaneous

Maximum pending reads: 0

Fixed read latency (cycles): 0

Timing:

TODOS DESACTIVADOS

Interface Address Type

Set master interface address type to symbols or words: WORDS

VHDL BFM

VHDK BFM ID: 0

- Pulsando en **Finish,** se volverá a la ventana de contenidos del sistema.
- Conectar el puerto `clk` del *mm_master_bfm_0* con el puerto `clk` del *clk_0*.
- Conectar el puerto `clk_rst` del *mm_master_bfm_0* con el puerto `clk_rst` del *clk_0*.
- Conectar el puerto `m0` del *mm_master_bfm_0* con el puerto `avalon_slave` del *displays7seg*.

22 En estos momentos, ya tenemos definido el sistema para verificación de nuestro diseño (Figura 16). Ahora tenemos que generar los ficheros de simulación necesarios. Antes de ello hacemos **File > Save**

23 El último paso en Platform Designer es generar los ficheros de simulación. Para ello, nos vamos a **Generate > Generate HDL…** y la configuramos con los siguientes parámetros (Figura 17):

Synthesis

- **Create HDL design files for synthesis**: None
- **Create block symbol file:** Desactivado

Simulation

- **Create simulation model**: None

Pulsamos Cancel

24 Nos vamos a **Generate > Generate Testbench System…** y la configuramos con los siguientes parámetros (Figura 17):

Tesbench System

- **Create testbench Platform Designer system**: Standard, BFMs for standard Platform Designer interconnect
- **Create testbench simulation model:** Verilog

Una vez configurado, pulsamos el botón **Generate,** generando los ficheros necesarios para llevar a cabo una simulación

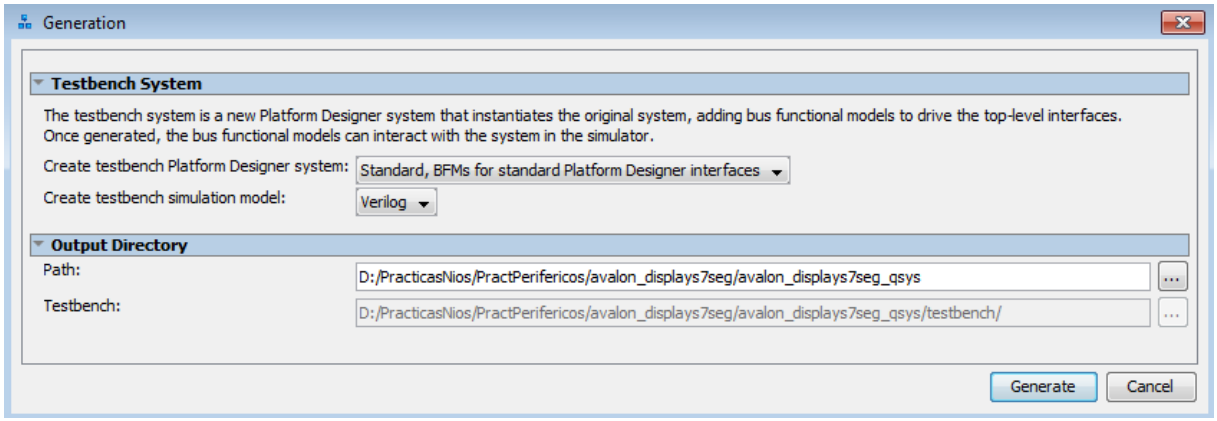


Figura 17: Pestaña Generate Testbench System del sistema Platform Designer

25 En esta fase, disponemos ya de ficheros para poder simular el sistema. Esto lo vamos a llevar a cabo en Modelsim.

6.3. Verificación del componente haciendo uso del software Modelsim

La validación del componente desarrollado junto con los BFM se lleva a cabo haciendo uso del software Modelsim. Los ficheros de simulación generados por Platform Designer servirán como base para generar un banco de pruebas. Platform Designer solo proporciona un modelo de simulación del sistema, el usuario tiene que crear un banco de pruebas (testbench) que genere distintos casos de test.

Durante este apartado, veremos cómo generar un banco de pruebas sencillo y cómo conectarlo con el modelo generado en Platform Designer. El banco de pruebas está compuesto de dos nuevos ficheros en verilog desarrollados por el usuario (top.sv y test_program.sv) y por todos los ficheros generados por Platform Designer cuyo top es avalon_displays7seg_qsys_tb.v.

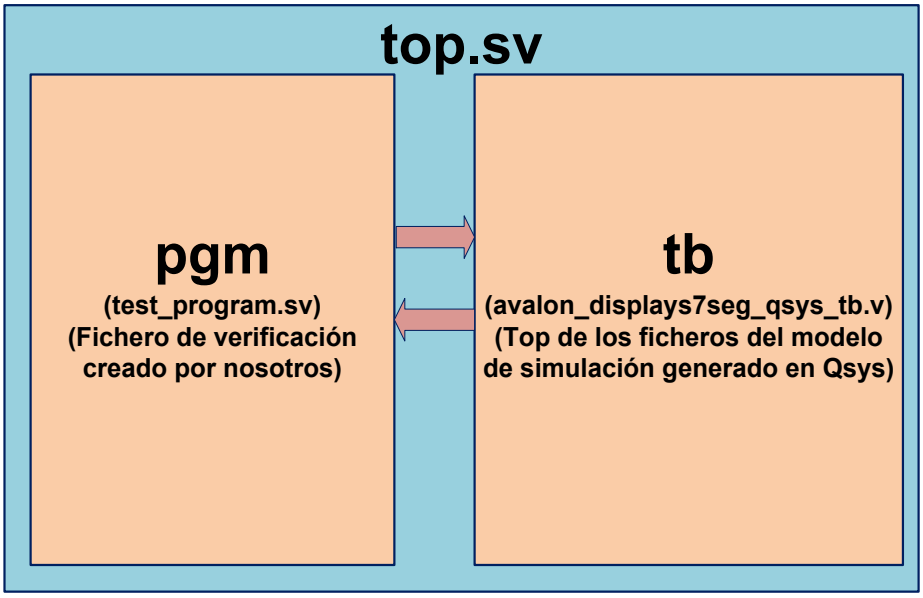


Figura 18: Esquema de la estructura de test

La estructura del test puede observarse en la Figura 18. El fichero top.sv es el top de la jerarquía. Es un fichero que generamos nosotros, dentro del cual tenemos dos módulos, el test_program.sv también creado por nosotros define todos los casos de test y actúa creando y leyendo estímulos del modelo de simulación. El modelo de simulación del sistema de Platform Designer creado está formado por una gran cantidad de ficheros verilog cuyo top de jerarquía es avalon_displays7se_qsys_tb.sv, que podemos encontrar en nuestro proyecto en la carpeta:

D:\PracticasNios\PractPerifericos\avalon_displays7seg\avalon_displays7seg_qsys\testbench\avalon_displays7seg_qsys_tb\simulation

Junto con el código del modelo, Platform Designer genera un script ***msim_setup.tcl*** para Modelsim para poder compilarlo.

Nosotros modificaremos el script para que en la compilación se incluyan los ficheros de test generados por nosotros.

El script generado para Modelsim, se encuentra en la siguiente carpeta en nuestro proyecto:

D:\PracticasNios\PractPerifericos\avalon_displays7seg\avalon_displays7seg_qsys\testbench\mentor

En este punto de la práctica, la intención no es desarrollar un banco de pruebas, la intención es que el alumno sepa integrarlo en la simulación. Durante la tarea, el alumno tendrá que crear su propio banco de pruebas. Por tanto, seguidamente voy a proporcionar el código correspondiente al top.sv y test_program.sv donde tenemos una verificación básica del componente.

```
module top();
      avalon_displays7seg_qsys_tb tb ();
```

```
      test_program pgm ();
endmodule
```

Código verilog del módulo de test top.sv

```
`timescale 1 ns / 1 ps


// console messaging level
`define VERBOSITY VERBOSITY_INFO


//BFM jerarquía
`define CLK tb.avalon_displays7seg_qsys_inst_clk_bfm_clk_clk
`define RST tb.avalon_displays7seg_qsys_inst_reset_bfm_reset_reset
`define BFM tb.avalon_displays7seg_qsys_inst.mm_master_bfm_0

`define AV_ADDRESS_W      3
`define AV_DATA_W         32

module test_program();

import avalon_mm_pkg::*;
import verbosity_pkg::*;

reg [`AV_DATA_W-1:0] datos_out;
int j=0;


initial
  begin
    set_verbosity(`VERBOSITY);
    `BFM.init();

    // Espera hasta que el reset esté inactivo
    wait(`RST == 1);
    #100
    // Escribe un dato en el esclavo*/
    avalon_write (3'h0,32'h00001234);
    repeat (5) @(posedge `CLK);
    avalon_write (3'h1,32'h00005678);
    $display("Operacion Escritura acabada");

    repeat (5) @(posedge `CLK);

    // Lee dato del esclavo y comprueba si es correcto
    avalon_read (3'h2,datos_out);

    $display("Datos leidos del IP: %h",datos_out);

    j++;

    $display("//////// ITERACION %d /////////",j);


    $stop();

  end

    // ============================================================
```

```
    // Tasks
    // ============================================================
    //
    // Avalon-MM single-transaction read and write procedures.
    //
    // ------------------------------------------------------------
    task avalon_write (
    // ------------------------------------------------------------

    input [`AV_ADDRESS_W-1:0] addr,
        input [`AV_DATA_W-1:0] data
    );
    begin
        // Construct the BFM request
        `BFM.set_command_request(REQ_WRITE);
        `BFM.set_command_idle(0, 0);
        `BFM.set_command_init_latency(0);
        `BFM.set_command_address(addr);
        `BFM.set_command_byte_enable('1,0);
        `BFM.set_command_data(data, 0);
        `BFM.set_command_burst_count(1);
        `BFM.set_command_burst_size(1);
        // Queue the command
        `BFM.push_command();

        // Wait until the transaction has completed
        while (`BFM.get_response_queue_size() != 1)
            @(posedge `CLK);

        // Dequeue the response and discard
        `BFM.pop_response();
    end
    endtask



    // ------------------------------------------------------------
    task avalon_read (
    // ------------------------------------------------------------
        input [`AV_ADDRESS_W-1:0] addr,
        output [`AV_DATA_W-1:0] data
    );
    begin
       // Construct the BFM request
      `BFM.set_command_request(REQ_READ);
      `BFM.set_command_idle(0, 0);
      `BFM.set_command_init_latency(0);
      `BFM.set_command_address(addr);
      `BFM.set_command_byte_enable('1,0);
      `BFM.set_command_data(0, 0);
      `BFM.set_command_burst_count(1);
      `BFM.set_command_burst_size(1);

     // Queue the command
      `BFM.push_command();

        // Wait until the transaction has completed
        while (`BFM.get_response_queue_size() != 1)
            @(posedge `CLK);

        // Dequeue the response and return the data
```

```
        `BFM.pop_response();
        data = `BFM.get_response_data(0);
    end
    endtask


endmodule
```

Código verilog del módulo de test test_program.sv

Durante este párrafo indicaremos cual tiene que ser el proceso para llevar a cabo la verificación con Modelsim

26 La simulación del sistema se va a llevar a cabo en el directorio donde Qsym ha generado el modelo de simulación:

D:\PracticasNios\PractPerifericos\avalon_displays7seg\avalon_displays7seg_qsys\testbench\avalon_displays7seg_qsys_tb\simulation

Allí crearemos los dos ficheros de simulación añadidos por nosotros:

- top.sv
- test_program.sv

Allí copiaremos el fichero script que vamos a modificar msim_setup.tcl
Por tanto, en estos momentos la carpeta de simulación queda como podemos ver en la Figura 19.

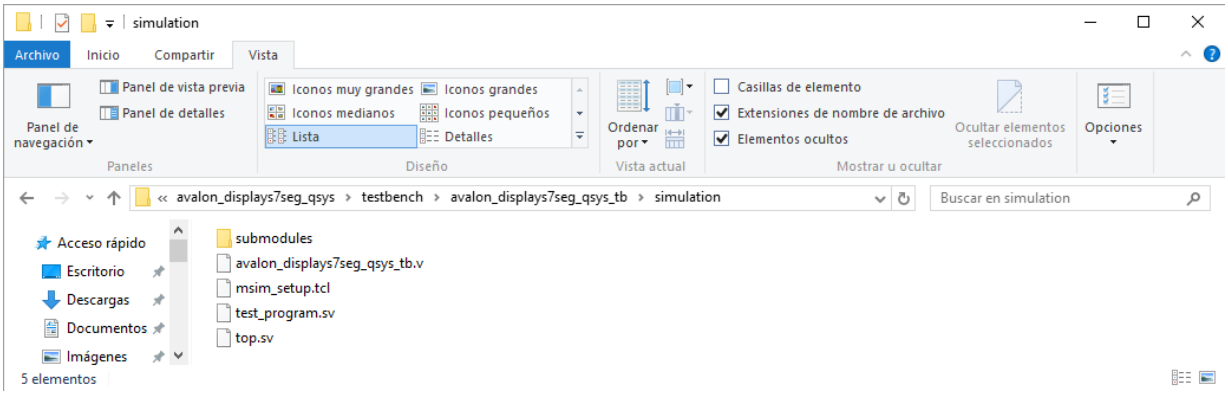


Figura 19: Carpeta del simulación del sistema Platform Designer

27 Vamos a comprobar que en el fichero **test_program.sv** donde tenemos los casos de test hemos definido correctamente el acceso a la señal de reloj, reset y al BFM. Abrimos el fichero test_program.sv y vemos que las etiquetas:

```
//BFM jerarquía
`define CLK tb.avalon_displays7seg_qsys_inst_clk_bfm_clk_clk
`define RST tb.avalon_displays7seg_qsys_inst_reset_bfm_reset_reset
`define BFM tb.avalon_displays7seg_qsys_inst.mm_master_bfm_0
```

apuntan correctamente a los módulos y señales siguiendo la jerarquía creada del testbench. Si dichas etiquetas están definidas de forma incorrecta, el banco de pruebas no funcionará pues test_program.sv no encontrará estos recursos.

28 El siguiente paso es editar el script de simulación **msim_setup.tcl que hemos copiado** en la carpeta de simulación para poder compilar el banco de pruebas con el modelo generado en Platform Designer. Para ello, abriendo el fichero hacemos los siguientes cambios:
Modificar las variables de entorno:

- **TOP_LEVEL_NAME : "top"**
  De esta forma, indicamos el fichero top de la jerarquía del banco de pruebas
- **QSYS_SIMDIR: "../.."**
  De esta forma, indicamos la ruta que nos permite realizar la compilación.

Debemos añadir librerías de compilación y ficheros de compilación.

Buscamos el fichero a compilar:

```
eval  vlog $USER_DEFINED_VERILOG_COMPILE_OPTIONS $USER_DEFINED_COMPILE_OPTIONS
"$QSYS_SIMDIR/avalon_displays7seg_qsys_tb/simulation/avalon_displays7seg_qsys_tb.v"
```

Tras él, añadiremos los ficheros de compilación nuevos que completan el banco de pruebas

```
eval vlog -sv -L  altera_common_sv_packages $USER_DEFINED_VERILOG_COMPILE_OPTIONS
$USER_DEFINED_COMPILE_OPTIONS  "$QSYS_SIMDIR/avalon_displays7seg_qsys_tb/simulation/test_program.sv"

eval vlog -sv $USER_DEFINED_VERILOG_COMPILE_OPTIONS $USER_DEFINED_COMPILE_OPTIONS
"$QSYS_SIMDIR/avalon_displays7seg_qsys_tb/simulation/top.sv"
```

Tras esto, guardamos el fichero y ya estamos listos para lanzar la simulación que nos permitirá verificar el correcto funcionamiento del diseño.

29 Ejecutamos el software Modelsim 10.5b software de simulación de Mentor graphics

30 Cambiamos el directorio de trabajo de Modelsim en **File > Change directory** indicando el directorio de simulación donde tenemos todos los ficheros:
D:\PracticasNios\PractPerifericos\avalon_displays7seg\avalon_displays7seg_qsys\testbench\avalon_displays7seg_qsys_tb\simulation

31 Nos vamos a la consola de trabajo de Modelsim y cargamos el script de compilación antes editado ejecutando el comando:
do msim_setup.tcl

32 Ejecutamos el script cargado desde la consola ejecutando el comando:
ld
Tras esto, se realiza la compilación de todos los ficheros del banco de pruebas y se carga la simulación pero todavía no se ejecuta.
Si existe algún error de compilación en el código, en este paso Modelsim informa de ello. El usuario tendrá que corregirlo antes de poder lanzar de nuevo la compilación (comando **ld**)

33 Si la simulación se ha cargado correctamente en el Modelsim veremos que en la pestaña **sim** podemos desplegar toda la jerarquía del testbench. Como vemos en la Figura 19, podemos bajar en la jerarquía del diseño hasta alcanzar los módulos diseñados por nosotros, en concreto sus instancias:

1. **displays7seg**: como top de nuestro diseño.
2. **u1_av_sl_MM**: como instancia del interface Avalon MM diseñado
3. **dec0 a dec7**: como las 8 instancias de los decodificadores

34 Si seleccionamos cualquiera de los módulos de la pestaña **Sim,** en la pestaña **Objects** aparecen todas las señales que definimos en el código verilog del diseño. En la Figura 19, se pueden ver las señales de nuestro módulo **displays7seg**

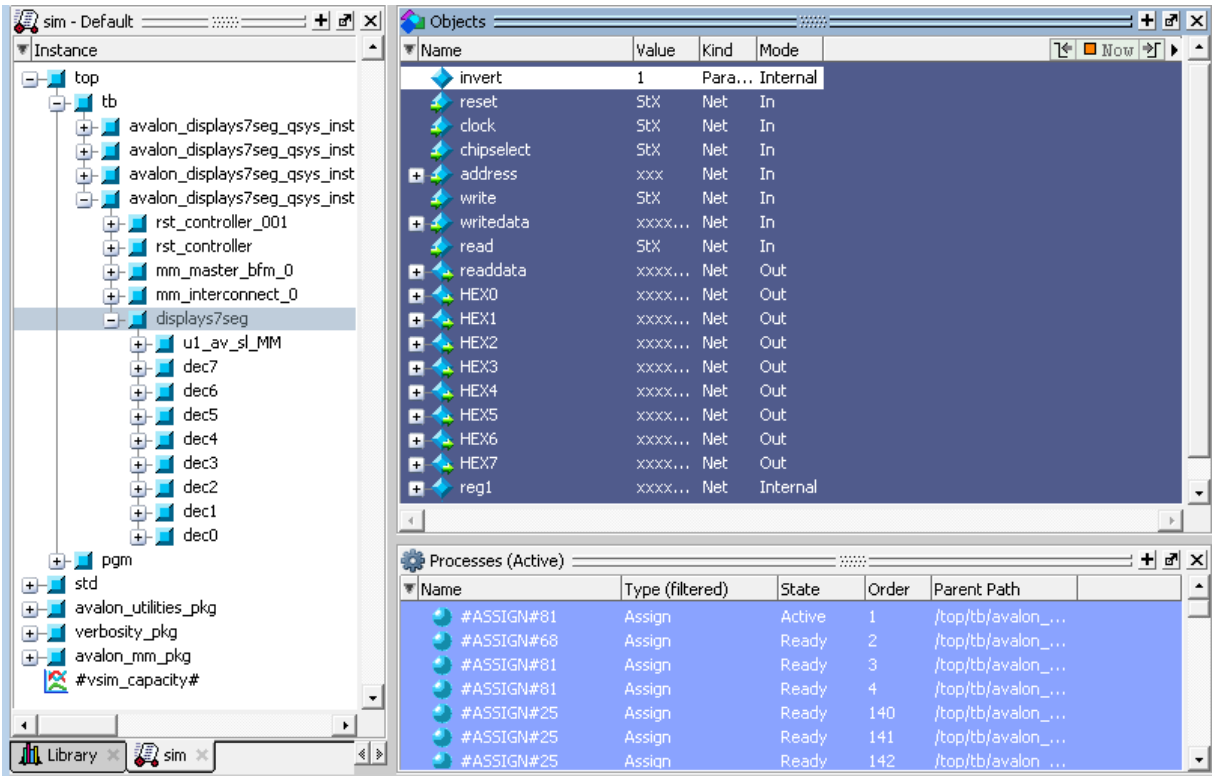


Figura 20: Pestaña Sim y Objects en Modelsim tras cargar la simulación

35 Si seleccionamos cualquiera de las señales de la pestaña Objects (Figura 20), pulsando el botón derecho del ratón podemos seleccionar la opción **Add Wave** añadiendo las formas de onda en la pestaña **Wave.** Cuando ejecutemos la simulación, veremos en esta pantalla las formas de onda y su comportamiento ante los distintos casos de test generados en la simulación

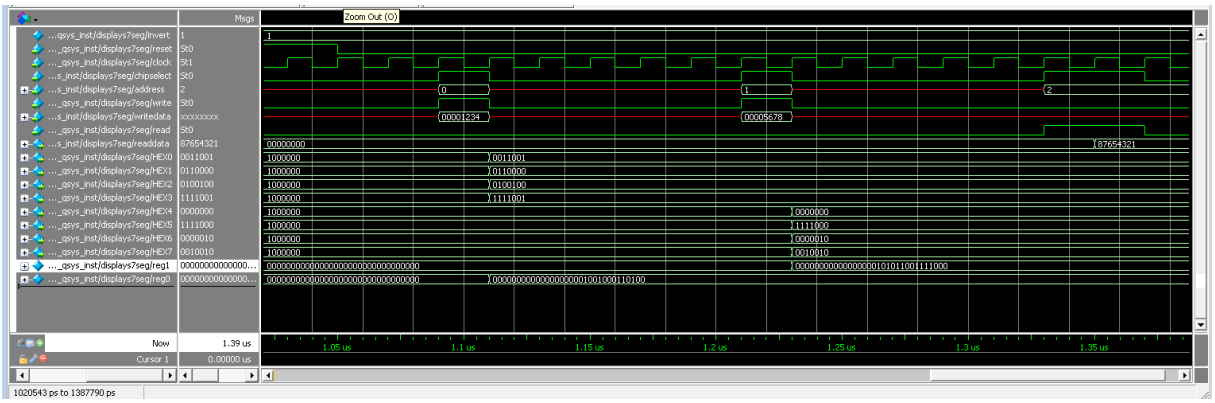


Figura 21: Pestaña Wave en Modelsim tras ejecutar la simulación

36 Para correr la simulación, vamos a la consola y ejecutamos **run –all** y la simulación se ejecutará hasta que en el código verilog del testbench encuentre una sentencia **$stop**. Podemos ejecutar un tiempo determinado de simulación, por ejemplo **run 1500 ns**.
Haciendo un zoom out sobre las formas de onda con los iconos con forma de lupa, podremos ver toda la simulación (Figura 21). Por la consola de simulación, aparecerán mensajes generados por el banco de pruebas.
Esto es lo que usaremos para verificar que nuestro diseño está funcionando correctamente.

37 Si queremos añadir una nueva señal, para verificar su comportamiento seleccionamos el módulo al que pertenece en la pestaña **Sim** y en la pestaña **Objects** seleccionamos la señal y con botón derecho **Add Wave.** Hay que volver a ejecutar la simulación. Para eso, ejecutaremos el comando **restart** y tras ello ejecutaremos nuevamente la simulación **run -all**

De esta forma, verificaremos que nuestro módulo se comporta como esperamos.

38 Cualquier error detectado en el código verilog deberá ser corregido y vuelto a verificar. El nuevo cambio realizado deberá actualizarse siguiendo el proceso:

1. Cambiaremos el código verilog
2. Actualizaremos el componente en Platform Designer
3. Volveremos a crear el modelo de simulación generado en Platform Designer.
4. Tras ello volveremos a ejecutar los scripts de Modelsim.

39 Una vez consideremos el diseño libre de errores podemos considerar el diseño listo para ser usado en un sistema basado en Nios II

7. Integración del nuevo componente en un Sistema en Nios II

En el sistema Platform Designer donde hemos generado nuestro componente tenemos los ficheros necesarios para importarlo a un nuevo proyecto Platform Designer:

- En la carpeta **display7seg** tenemos todos los ficheros fuente del nuevo componente
- Platform Designer, al integrar el componente ha generado un fichero que describe todo el diseño como un elemento de librería de Platform Designer **avalon_displays7seg_hw.tcl**

En el proyecto en que queramos hacer uso del componente, deberemos crear en la carpeta raíz del proyecto una nueva carpeta llamada **ip**. Solo con copiar la carpeta **display7seg** y el **avalon_displays7seg_hw.tcl** en la carpeta **ip** ya conseguimos importar el nuevo componente al nuevo proyecto. La próxima vez que creemos o abramos un proyecto Platform Designer en el nuevo proyecto nos aparecerá la librería **myAvlonIPs** con nuestro componente.

En la carpeta **ip** podemos ir añadiendo nuestros componentes. Es importante que en un proyecto no se haga uso de varios componentes que tengan ficheros en verilog con el mismo nombre. En un proyecto en verilog no puede haber módulos con descripciones verilog diferentes que tengan el mismo nombre, es decir, verilog no permite la sobrecarga de módulos.

El uso del componente es el mismo que cualquiera de los componentes ya usados durante prácticas previas. Lo único a tener en cuenta a la hora de realizar el software en C es que las direcciones de acceso al componente y su funcionalidad son las que hemos definido nosotros en el código verilog.

Teniendo en cuenta esto, sería sencillo modificar cualquier proyecto basado en Nios II que haga uso de los displays de siete segmentos para introducir el nuevo componente eliminando del software en C del Nios II el código que realiza la conversión de hexadecimal a siete segmentos pues esto es lo que hará nuestro diseño. En Quartus, en la instancia del qip generadado por el proyecto platform designer, deberemos conectar las salidas de la ip a los 6 siete segmentos disponibles en la DE1-SoC. La ip está diseñada para poder usar hasta 8 displays siete segmentos, dos de las salidas de la ip quedarán sin conectar.

Como ejercicio propongo modificar uno de los proyectos llevados a cabo anteriormente en la asignatura para poder añadir este nuevo componente eliminando del sistema los componentes que hacía uso de los displays de siete segmentos.

8. TAREA: Diseño de un Timer propio en Verilog para Nios II

El componente a desarrollar en verilog es un timer que puede generar una interrupción a final de cuenta, que podrá sustituir al timer ya usado en el proyecto del reloj. El timer poseerá las siguientes características:

- La cuenta del timer será de 32bits

- El componente será programable a través de un bus Avalon_MM. El componente se comportará como un Avalon-MM slave.
- El final de cuenta será programable.
- El inicio y parada del componente serán programables.
- Si el timer no es parado por el usuario, cuando éste llega al final de cuenta programado el timer vuelve a cero y continúa contando.
- El timer posee una señal de salida de interrupción. Cuando el timer llega a final de cuenta, el timer activará la línea de interrupción siempre que la interrupción esté habilitada.
- El usuario deberá desactivar la interrupción para que esta se pueda generar de nuevo.
- El usuario puede acceder al valor actual la cuenta leyendo en una dirección de memoria del timer.
- El timer poseerá un umbral programable independiente de valor programable del final de cuenta. Cuando el umbral sea superado por el timer, el módulo activará una señal hacia el exterior del sistema Platform Designer. Esa señal será vinculada a un led de la placa DE1-SoC para comprobar su correcto funcionamiento.
- El diseñador deberá incluir los registros de lectura, escritura y lectura/escritura mapeados en memoria que considere necesarios para desarrollar toda la funcionalidad solicitada. Definirá sus propias direcciones de memoria a las cuales se accederá a través del bus Avalon-MM
- El tamaño mínimo de bits para las direcciones será de 3bits, aunque el usuario use menos registros de los direccionables con 3bits

8.1. Descripción de los interfaces que poseerá el módulo

El módulo poseerá los siguientes interfaces:

- Interface de entrada de reloj (**Clock Input**).
- Interface de entrada de reset (**Reset Input**).
- Interface Conduit para sacar del sistema el bit activado por umbral programable **(Conduit).**
- Interface de interrupción (**Interrupt Sender**)
  La señal de interrupción será activa a nivel alto hasta que el usuario la desactive.

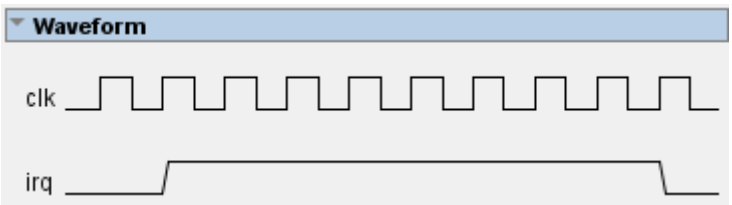


Figura 22: Temporización de la interrupción

  Para validar la interrupción utilizaremos el BFM: Altera **Avalon Interrupt Sink**.
  Al valor de la señal de interrupción se accederá con el DEFINE adecuado en el test_program.sv desarrollado para esta aplicación. El usuario deberá introducir el DEFINE adecuado en el test_progrma.v
- Interface Avalon-MM-slave (**Avalon Memory Mapped Slave**)
  Usaremos un interface con la misma temporización que la del ejemplo del decodificador.

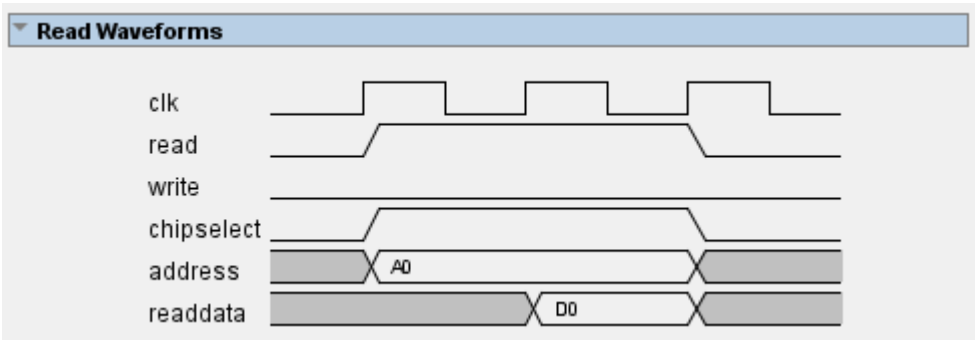


Figura 23: Escritura sobre Avalon-MM slave

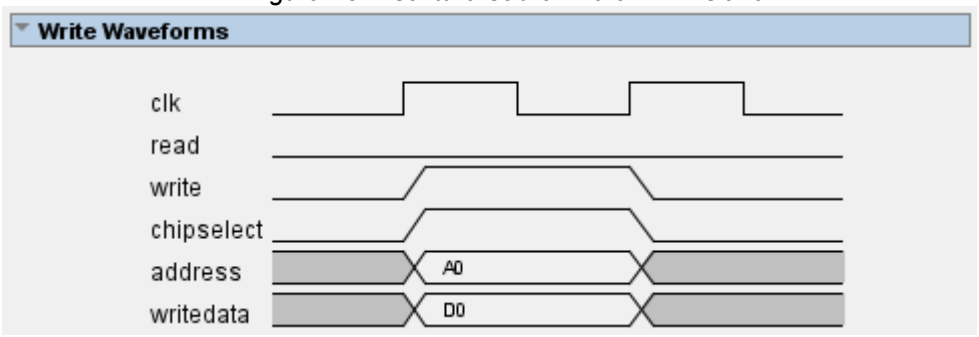


Figura 24: Lectura sobre Avalon-MM slave

Para validar el BFM del interface Avalon MM slave, usaremos el mismo BFM del decodificador desarrollado anteriormente **Altera Avalon-MM Master BFM** y configurado de la misma forma.

8.2. Descripción de la tarea, entregables y valoración

La tarea constará de las 3 fases del desarrollo de un nuevo componente para Nios II:

1. Generación del nuevo componente creando el proyecto Quartus II
2. Verificación del componente creando un sistema Platform Designer con BFM para Avalon-MM y para la interrupción. La verificación se llevará a cabo en Modelsim y deberá cubrir toda la funcionalidad del componente.
3. Integración del timer en el proyecto del reloj de la tarea anterior, sustituyendo al timer del sistema y conectando la salida por umbral a uno de los LEDs del sistema. El diseño deberá ser verificado sobre la placa DE1-SoC. El alumno deberá modificar el software en C para poder acceder al nuevo componente y validar la nueva funcionalidad. Habrá que comprobar el correcto funcionamiento de la interrupción.

Los alumnos deberán enviar un archivo comprimido por cada uno de los proyectos de las tres fases junto con un documento donde se especifiquen las carpetas correspondientes a cada uno de los proyectos y el mapa de registros del componente desarrollado y su funcionalidad. Los trabajos serán realizados en grupo, pero todos los alumnos deben realizar la entrega. En el documento adjunto también deberán indicarse las personas con las que se realizó el proyecto. La documentación aportada por cada uno de los miembros de un equipo podrá ser la misma. La documentación y proyectos entregados por gente no perteneciente al mismo grupo deberán ser diferentes.

La calificación de la tarea será:

- FASE1: Generación del componente (40% de la nota de la tarea)
- FASE2: Sistema Platform Designer de verificación y verificación con Modelsim (30% de la nota de la tarea)
- FASE3: Integración del nuevo componente en el proyecto anterior del reloj en su versión más sencilla (30% de la nota de la tarea)

RECOMIENDO UTILIZAR COMO REFERENCIA PARA LA COMUNICACIÓN AVALON-MM Y PARA LA VERIFICACIÓN LOS CÓDIGOS PROPORCIONADOS DEL DECODIFICADOR DE HEXADECIMAL A DISPLAYS DE SIETE SEGMENTOS